Кейс «Лаборатория протоколов» · Мадригал

protoledger

Журнал исследования протокола: от записей трафика к проверяемому, повторно применимому и переносимому описанию обмена.

локально и офлайн

окно · браузер · командная строка

Rust + Nuxt

github.com/protoledger/protoledger

Проблема

Подключить систему к устройству без документации на протокол — обычная задача интеграции: контроллеры, измерительные приборы, IoT, устаревшие сервисы.

Сегодня это Wireshark, hex-редактор, заметки и одноразовые скрипты.

Что ломается

- Выводы **не привязаны к байтам** конкретной записи
- Догадка выглядит так же, как установленный факт
- На новой записи всё проверяется **заново руками**
- Коллеге передаётся результат, но **не основания**
- Дефекты захвата (потери, повторы, offload) путают выводы

Решение: цикл исследования в одном инструменте

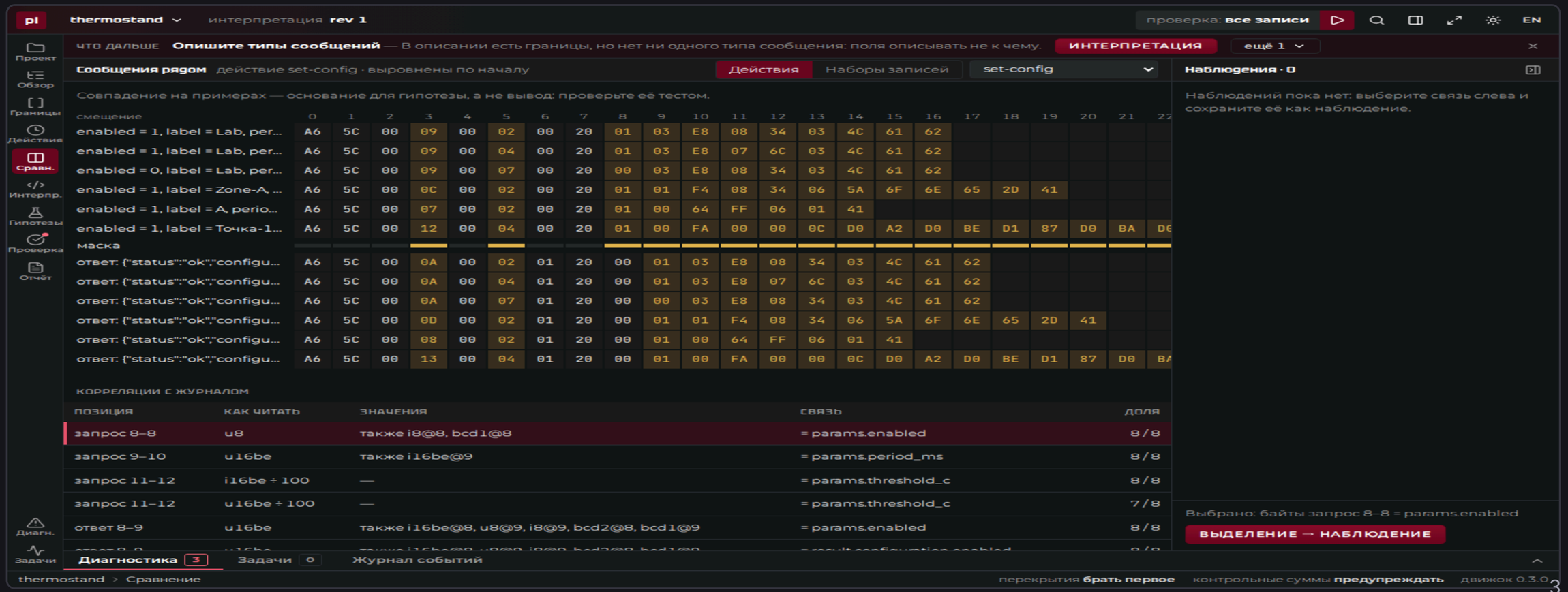
Записи
сборка TCP, дефекты явно

Исследование
границы, обмены, сравнение

Описание
правило · гипотеза · неизвестно

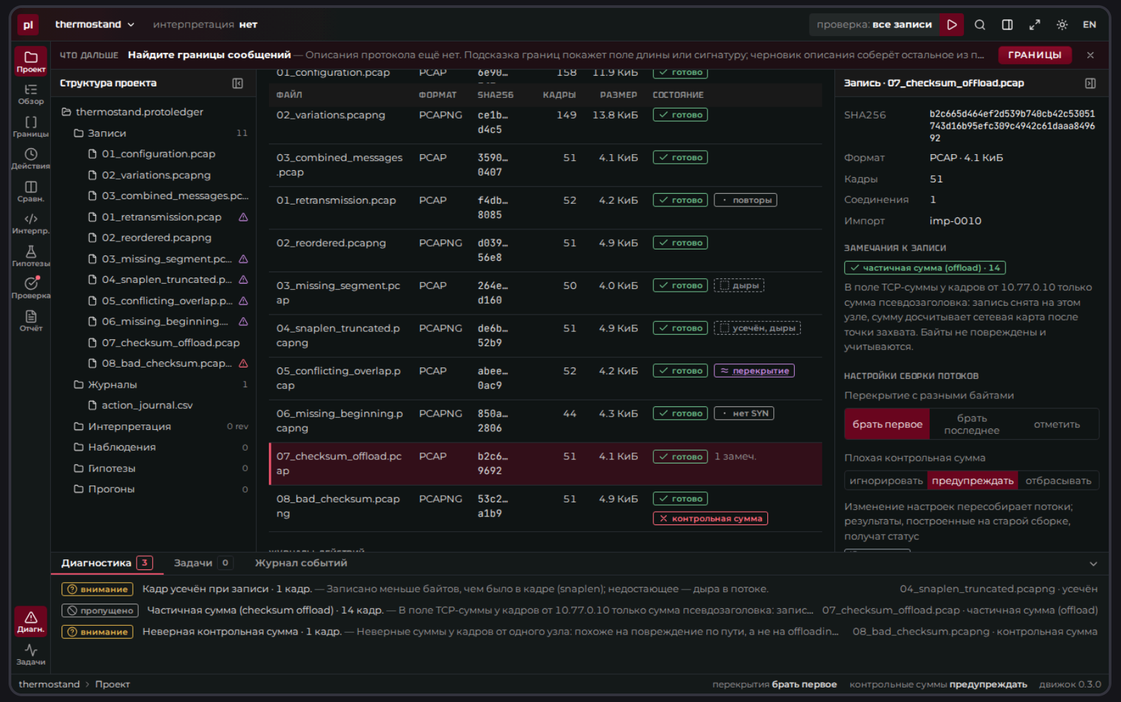
Проверка
корпус, контрпримеры

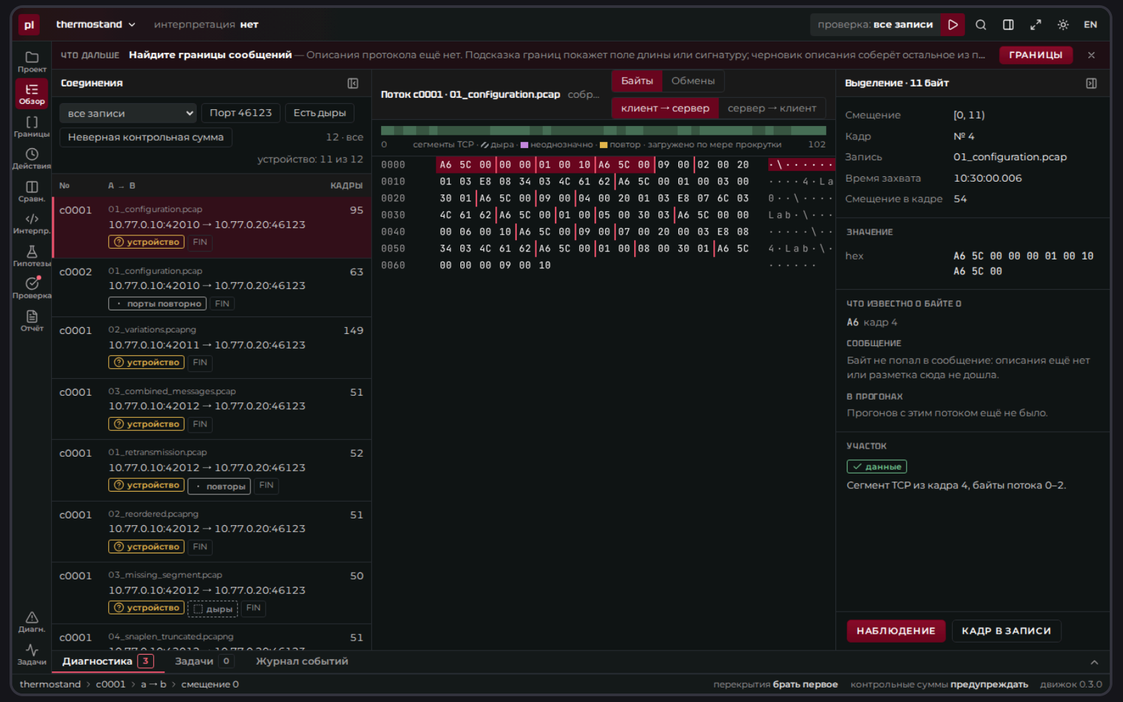
Передача
отчёт, YAML, папка проекта



Демо 1. Записи и потоки

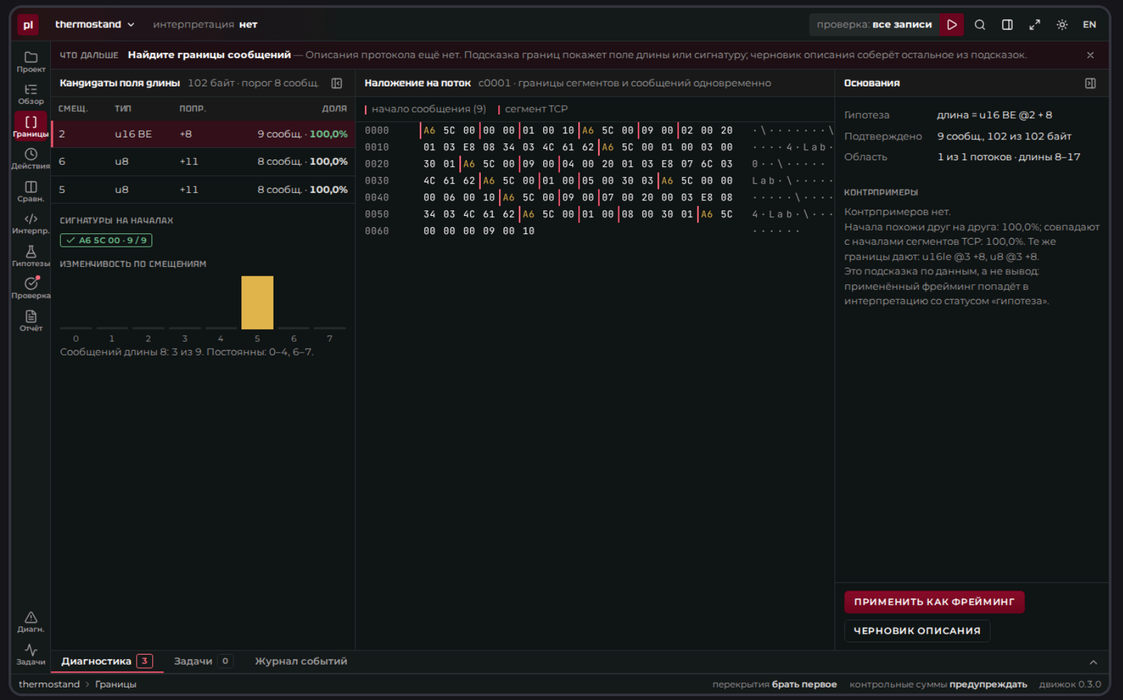
11 записей «Термостенда» и журнал из 109 действий; колонки журнала угадываются сами

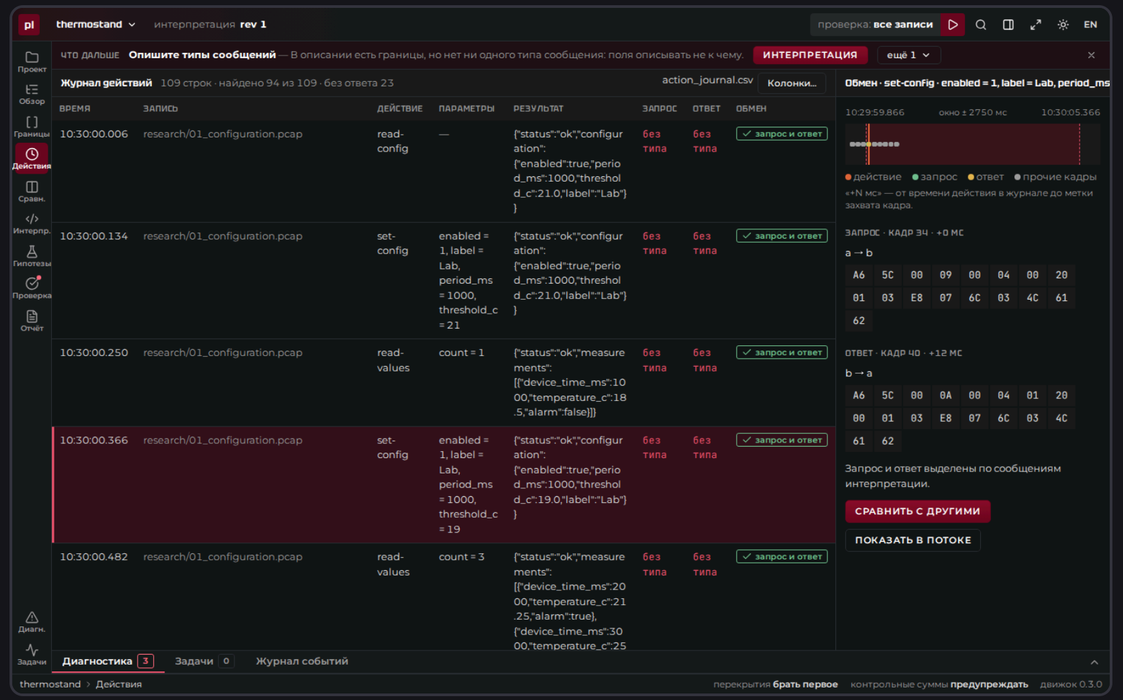




Повторы не задваиваются, дыра остаётся дырой, перекрытие с разными байтами — «неоднозначно», частичная контрольная сумма распознана как offload. Каждый байт — с кадром и смещением.

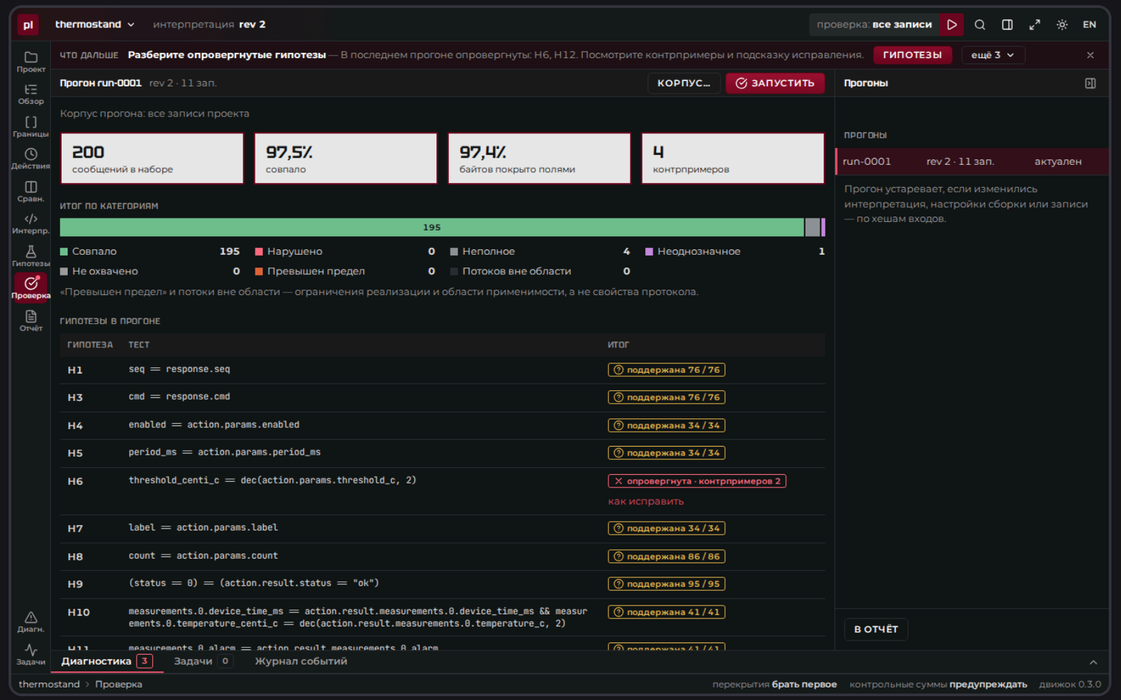
Демо 2. Границы, обмены, связи с журналом



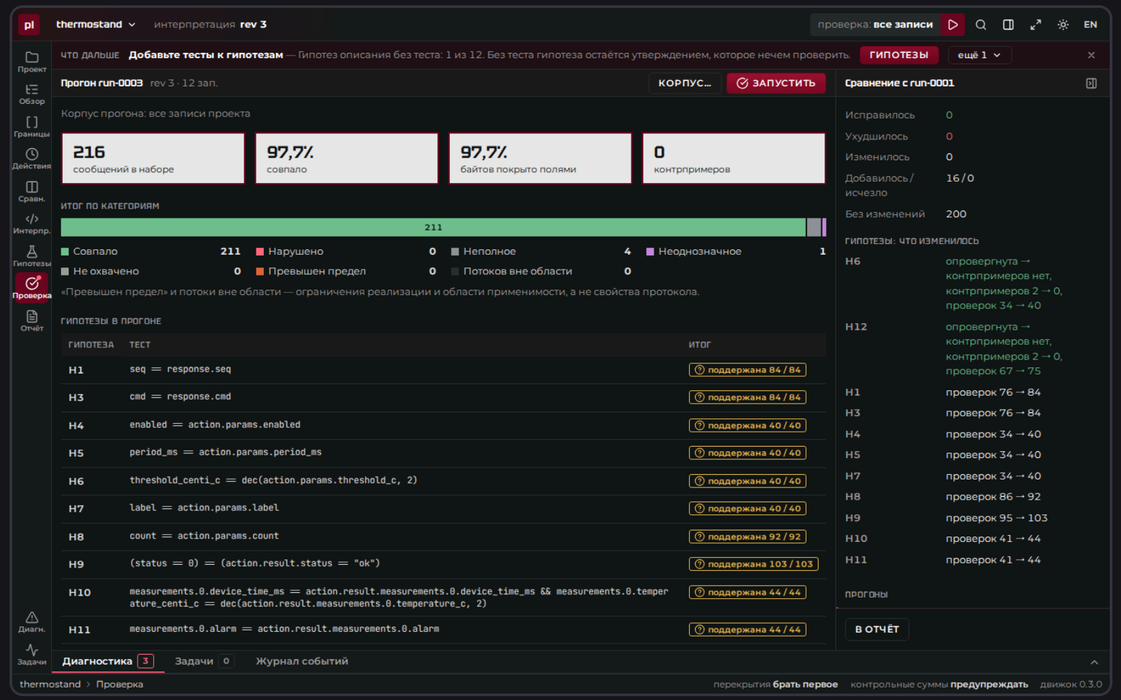


Поле длины u16 BE @2 +8 подтверждено на 100 % сообщений, сигнатура A6 5C. Для действия set-config с порогом 19 найдены запрос и ответ: 07 6C = 1900. Это подсказки с основаниями, а не выводы.

Демо 3. Ошибка → контрпример → уточнение



Порог прочитан как **u16**: 200 сообщений, гипотезы H6 и H12 опровергнуты на пороге −2,5 °C (65286 ≠ −250), со строкой журнала.



Исправлено на **i16** — новая ревизия, прогон по всему корпусу: 216 сообщений, 0 контрпримеров; сравнение прогонов показывает, что изменилось.

Как это работает

```
SPA (Nuxt) — в браузере или окне Tauri
        |  HTTP JSON /api · SSE · только 127.0.0.1
        |  Host/Origin + токен сессии
движок protoledger (Rust, один бинарник)
  pl-server    API, защита, встроенная SPA
  pl-app       сценарии, реестр задач
  pl-capture   pcap/pcapng → индекс кадров
  pl-reassembly TCP: потоки, дыры, перекрытия
  pl-actions   журнал действий (CSV)
  pl-interp    описание протокола, выражения
  pl-verify    прогоны, контрпримеры
  pl-report    отчёт MD / HTML
  pl-project   папка проекта, атомарная запись
        |
<имя>.protoledger/  YAML/JSON + копии записей
```

Один движок — три оболочки

Окно приложения (.msi, .deb, AppImage), браузер (`docker compose up` или бинарник `serve`), командная строка: `apply`, `verify`, `report`, `draft`.

Без базы данных

Состояние — папка проекта: текстовые YAML/JSON и неизменные копии записей с sha256. Перенос — копирование папки, `protoledger verify` воспроизводит прогоны.

Достоверность и воспроизводимость

Знание со статусом

- Поле — правило, гипотеза или неизвестно
- Гипотеза — «поддержана k из n», не «доказана»
- Каждый вывод — якорь на байты записи

Честные категории

- Свойства протокола: нарушено, не охвачено
- Дефекты записи: неполное, неоднозначное
- Ограничения реализации: не поддержано, превышен предел

Воспроизводимо

- Ревизии описания хранятся все
- Прогон подписан хешами входов: устаревший виден
- Детерминированный вывод: CLI ≡ интерфейс

Безопасность

- Только 127.0.0.1, проверка Host и Origin, токен сессии, CSP
- Без `unsafe`, без `unwrap` на данных

Злые входы

- 93 «злых» файла в корпусе, 5 целей фаззинга
- Битый кадр — диагностика, а не падение

Проверки в CI

- 419 тестов движка, fmt и clippy без предупреждений, аудит лицензий
- 16 эталонов, демо по кнопкам в браузере

Результаты на данных кейса

«ТЕРМОСТЕНД», НАБОР ОРГАНИЗАТОРОВ	
Записей / действий журнала	**11 / 109**
Сообщений после уточнения, контрпримеров	**216 / 0**
Дополнительная запись без правки описания	**16 из 16**
Черновик описания без знаний о протоколе	**16 из 16**
Диагностические записи: дефект распознан	**8 из 8**
Modbus и MQTT — тот же движок, без кода о них	**эталоны в CI**
Чужая запись 4SICS (Modbus/TCP)	**19 из 19**

ПРОФИЛЬ Т3: 100 МИБ, 250 ТЫС. КАДРОВ, 1000 СОЕДИНЕНИЙ, ОТВЕТЫ ДО 64 КИБ; 4 VCPU / 8 ГИБ	
Разбор записи (CLI)	**0,20 с**
Импорт в проект: копия, sha256, сборка	**0,80 с**
Прогон 235 тыс. сообщений с гипотезами	**0,78 с**
Отчёт	**0,02 с**
Пик памяти движка	**200 МиБ**

Ограничения и развитие

Сейчас — честно

- Область Т3: Ethernet + IPv4 + TCP; IPv6, UDP и фрагменты IP — пропускаются с диагностикой
- Шифрованный трафик не разбирается — показывается как неописанные байты
- Запись читается в память целиком (предел 1 ГиБ)
- Гипотеза не становится доказанной: проверена только на наблюдавшихся значениях
- Установщики — Windows и Linux

Дальше

- IPv6 и UDP — новые слои того же конвейера
- Чтение записи частями для файлов больше памяти
- Эксперименты со стендом из приложения: конструктор сообщений
- Двойник устройства по выученным парам «запрос → ответ»
- Экспорт парсера: Wireshark уже есть, далее Kaitai Struct

Команда и ссылки

sleepyhead-dev	Backend, DevOps
OddS-Programming	Backend
rinna32	Frontend
sky1768205	Frontend

Репозиторий:
github.com/protoledger/protoledger

Релиз v0.3: установщики, бинарники, запись экрана демо — страница Releases

Запуск: `git clone --recursive` → `docker compose up --build` → localhost:8080

Как пользоваться: docs/guide.md · **проверка ТЗ:** docs/requirements.md